The Journal of Open Source Software

StateCounter: state tracking for composite objects in complex combinatorial libraries

Zhihan Liu[1] **and Justin B. Kinney**[1¶]

1 Cold Spring Harbor Laboratory, Cold Spring Harbor, NY, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

StateCounter is a Python library for decomposing the state of a composite object into the states of its components. Many scientific computing tasks involve creating complex combinatorial structures—for example, a library of DNA sequences generated by combining different mutation types, positions, and barcodes. Given the index of an item in such a structure, one often needs to determine which specific choices were made to produce it. StateCounter solves this problem by letting users build a directed acyclic graph (DAG) of counters using operations like Cartesian products and disjoint unions. When the user sets the state of a derived counter, StateCounter automatically propagates this value through the DAG, computing the corresponding state of every component counter. We developed StateCounter to support DNA library generation, but it addresses a general need applicable to experimental design, parameter sweeps, combinatorial testing, and other domains.

Statement of Need

Given the index of an item in a complex combinatorial structure, how does one trace it back to the states of its components? This problem arises whenever composite objects are constructed from multiple independent choices—and the answer matters for random access (generating item n without iterating through items 0 to $n-1$), systematic naming (encoding combinatorial origin in each item's identifier), and reproducibility (tracing experimental results back to specific parameter combinations).

We initially developed StateCounter as part of PoolParty, a Python package for generating complex DNA sequence libraries. We subsequently released StateCounter as a standalone package because it addresses a much more general need; PoolParty now includes StateCounter as a dependency.

Fig. 1 illustrates the motivating application: a 40-sequence DNA library comprising different types of variants of a cis-regulatory element (CRE). Each variant is created using a specific mutational process—point mutations, deletions, or insertions—applied at specific positions, and each variant is paired with one of two barcodes. The problem we faced was this: given a value n, how do we compute which mutations to introduce into the sequence and which barcode to attach?

DRAFT

state	name	seq
0	mut_0.v0	TCCCGACT<cre>GGAAAGCaGGCAaTGAGCACcaAGG</cre>ATTACGG<bc>tggcc</bc>AGATCGGA
1	mut_0.v1	TCCCGACT<cre>GGAAAGCaGGCAaTGAGCACcaAGG</cre>ATTACGG<bc>aaaat</bc>AGATCGGA
2	mut_1.v0	TCCCGACT<cre>GGAAAcCGGGCAGTGAGgACACAta</cre>ATTACGG<bc>gtggt</bc>AGATCGGA
3	mut_1.v1	TCCCGACT<cre>GGAAAcCGGGCAGTGAGgACACAta</cre>ATTACGG<bc>ggggt</bc>AGATCGGA
4	mut_2.v0	TCCCGACT<cre>GGAAAGaGGGCAGaGAGCACAatGG</cre>ATTACGG<bc>ctgac</bc>AGATCGGA
5	mut_2.v1	TCCCGACT<cre>GGAAAGaGGGCAGaGAGCACAatGG</cre>ATTACGG<bc>tgatg</bc>AGATCGGA
6	mut_3.v0	TCCCGACT<cre>GGAAcGCGGGCAGTctaCACACAGG</cre>ATTACGG<bc>taata</bc>AGATCGGA
7	mut_3.v1	TCCCGACT<cre>GGAAcGCGGGCAGTctaCACACAGG</cre>ATTACGG<bc>gaccc</bc>AGATCGGA
8	mut_4.v0	TCCCGACT<cre>GcAAAGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>caaaa</bc>AGATCGGA
9	mut_4.v1	TCCCGACT<cre>GcAAAGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>gggcg</bc>AGATCGGA
10	del_0.v0	TCCCGACT<cre>-----GCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>tcctt</bc>AGATCGGA
11	del_0.v1	TCCCGACT<cre>-----GCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>tcgtg</bc>AGATCGGA
12	del_1.v0	TCCCGACT<cre>GGAAA-----CAGTGAGCACACAGG</cre>ATTACGG<bc>tggct</bc>AGATCGGA
13	del_1.v1	TCCCGACT<cre>GGAAA-----CAGTGAGCACACAGG</cre>ATTACGG<bc>aggtg</bc>AGATCGGA
14	del_2.v0	TCCCGACT<cre>GGAAAGCGGG-----AGCACACAGG</cre>ATTACGG<bc>ccccg</bc>AGATCGGA
15	del_2.v1	TCCCGACT<cre>GGAAAGCGGG-----AGCACACAGG</cre>ATTACGG<bc>tatgc</bc>AGATCGGA
16	del_3.v0	TCCCGACT<cre>GGAAAGCGGGCAGTG-----ACAGG</cre>ATTACGG<bc>ggccg</bc>AGATCGGA
17	del_3.v1	TCCCGACT<cre>GGAAAGCGGGCAGTG-----ACAGG</cre>ATTACGG<bc>ggctc</bc>AGATCGGA
18	del_4.v0	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCAC-----</cre>ATTACGG<bc>ctcag</bc>AGATCGGA
19	del_4.v1	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCAC-----</cre>ATTACGG<bc>gaact</bc>AGATCGGA
20	ins_0.site_0.v0	TCCCGACT<cre>aaaaaGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>ctcat</bc>AGATCGGA
21	ins_0.site_0.v1	TCCCGACT<cre>aaaaaGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>taagc</bc>AGATCGGA
22	ins_0.site_1.v0	TCCCGACT<cre>ttttGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>gatct</bc>AGATCGGA
23	ins_0.site_1.v1	TCCCGACT<cre>ttttGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>tgata</bc>AGATCGGA
24	ins_1.site_0.v0	TCCCGACT<cre>GGAAAaaaaaCAGTGAGCACACAGG</cre>ATTACGG<bc>gctat</bc>AGATCGGA
25	ins_1.site_0.v1	TCCCGACT<cre>GGAAAaaaaaCAGTGAGCACACAGG</cre>ATTACGG<bc>aggtc</bc>AGATCGGA
26	ins_1.site_1.v0	TCCCGACT<cre>GGAAAttttCAGTGAGCACACAGG</cre>ATTACGG<bc>tgtat</bc>AGATCGGA
27	ins_1.site_1.v1	TCCCGACT<cre>GGAAAttttCAGTGAGCACACAGG</cre>ATTACGG<bc>tacga</bc>AGATCGGA
28	ins_2.site_0.v0	TCCCGACT<cre>GGAAAGCGGGaaaaaAGCACACAGG</cre>ATTACGG<bc>ggttc</bc>AGATCGGA
29	ins_2.site_0.v1	TCCCGACT<cre>GGAAAGCGGGaaaaaAGCACACAGG</cre>ATTACGG<bc>cctac</bc>AGATCGGA
30	ins_2.site_1.v0	TCCCGACT<cre>GGAAAGCGGGttttAGCACACAGG</cre>ATTACGG<bc>actgc</bc>AGATCGGA
31	ins_2.site_1.v1	TCCCGACT<cre>GGAAAGCGGGttttAGCACACAGG</cre>ATTACGG<bc>tgtac</bc>AGATCGGA
32	ins_3.site_0.v0	TCCCGACT<cre>GGAAAGCGGGCAGTGaaaaaACAGG</cre>ATTACGG<bc>ttccc</bc>AGATCGGA
33	ins_3.site_0.v1	TCCCGACT<cre>GGAAAGCGGGCAGTGaaaaaACAGG</cre>ATTACGG<bc>gatac</bc>AGATCGGA
34	ins_3.site_1.v0	TCCCGACT<cre>GGAAAGCGGGCAGTGttttAACAGG</cre>ATTACGG<bc>cgggt</bc>AGATCGGA
35	ins_3.site_1.v1	TCCCGACT<cre>GGAAAGCGGGCAGTGttttAACAGG</cre>ATTACGG<bc>taaag</bc>AGATCGGA
36	ins_4.site_0.v0	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCACaaaaa</cre>ATTACGG<bc>ttgtt</bc>AGATCGGA
37	ins_4.site_0.v1	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCACaaaaa</cre>ATTACGG<bc>aatat</bc>AGATCGGA
38	ins_4.site_1.v0	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCACttttt</cre>ATTACGG<bc>ttcag</bc>AGATCGGA
39	ins_4.site_1.v1	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCACttttt</cre>ATTACGG<bc>tctct</bc>AGATCGGA

Figure 1: Figure 1: A 40-sequence DNA library generated by PoolParty. Each sequence comprises a cis-regulatory element (CRE) flanked by constant regions and tagged with a unique barcode. The CRE region (red uppercase) is modified in three ways: random point mutations (states 0–9), tiled 5 nt deletions (states 10–19, shown as dashes), or tiled 5 nt insertions (states 20–39). Mutations and insertions are shown in bold blue lowercase. Each CRE variant is paired with two different barcodes (v0 and v1). XML-like tags (gray) demarcate the CRE and barcode regions.

State of the Field

The state decomposition problem arises across many domains—experimental design matrices, parameter sweeps, combinatorial software testing (PLACEHOLDER, PLACEHOLDER), and procedural content generation—yet no general-purpose tool exists for solving it. Python provides several utilities for working with combinatorial structures, but none address the state propagation problem that StateCounter solves. The `itertools` module (Python Software Foundation, 2024) provides `product` for Cartesian products and `chain` for concatenation, but these return iterators or tuples without any mechanism to trace a flat index back to component values. NumPy's `unravel_index` and `ravel_multi_index` functions (Harris et al., 2020) convert between flat and multi-dimensional indices, but only for rectangular array shapes; they cannot represent structures like disjoint unions or nested combinations of products and unions. Manual index arithmetic using `divmod` works for simple products but does not compose—each additional operation (slicing, shuffling, sampling) requires rewriting the index

logic from scratch. This gap persists because the problem appears trivial in simple cases; complexity emerges only when multiple operations are composed, by which point ad hoc solutions are deeply embedded in application code.

Software Design

StateCounter addresses this gap by constructing a directed acyclic graph (DAG) of counters. Users build the DAG by composing leaf counters into derived counters; when the user sets the state of a derived counter, StateCounter automatically propagates this value to all ancestor counters, computing the corresponding state of each. This enables arbitrary composition of products, stacks, slices, shuffles, samples, and splits while maintaining correct state tracking throughout.

The basic unit is a Counter, which represents an object that can take on a finite number of discrete states numbered from 0 to $n-1$. Users first create "leaf" counters specifying the number of states, then compose them using operations to build derived counters. The following operations are supported:

Operation	Description
`product`	Cartesian product; all parent counters active
`stack`	Disjoint union; one parent counter active at a time
`slice`	Select subset of states
`shuffle`	Randomly permute states
`sample`	Sample states with or without replacement
`split`	Divide into sub-counters by count or proportion
`repeat`	Cycle through states multiple times
`sync`	Synchronize counters in lockstep
`interleave`	Alternate between counters

All counters must be created within a `Manager` context. The Manager serves as a registry that tracks all counters created within a session, assigns unique identifiers, and provides convenience methods for bulk operations such as resetting all counters or generating iteration tables. While each counter independently tracks its own parent relationships, the Manager provides the global view needed for operations that span the entire collection.

Importantly, the computational graph constructed by StateCounter is a DAG, not necessarily a tree—the same counter can appear in multiple branches. This enables powerful patterns, such as using a single counter to synchronize choices across different parts of a composite structure. However, shared counters also introduce the possibility of conflicts: if counter `A` appears in both inputs to a `product`, then setting the product's state would require `A` to have two different values simultaneously. StateCounter automatically detects such conflicts at DAG construction time and raises an informative error, preventing subtle bugs that would otherwise arise from inconsistent state propagation.

DRAFT

The Journal of Open Source Software

Example

seq_counter	v_counter	cre_counter	mut_counter	del_counter	ins_counter	ins_position_counter	ins_site_counter
0	0	0	0	None	None	None	None
1	1	0	0	None	None	None	None
2	0	1	1	None	None	None	None
3	1	1	1	None	None	None	None
4	0	2	2	None	None	None	None
5	1	2	2	None	None	None	None
6	0	3	3	None	None	None	None
7	1	3	3	None	None	None	None
8	0	4	4	None	None	None	None
9	1	4	4	None	None	None	None
10	0	5	None	0	None	None	None
11	1	5	None	0	None	None	None
12	0	6	None	1	None	None	None
13	1	6	None	1	None	None	None
14	0	7	None	2	None	None	None
15	1	7	None	2	None	None	None
16	0	8	None	3	None	None	None
17	1	8	None	3	None	None	None
18	0	9	None	4	None	None	None
19	1	9	None	4	None	None	None
20	0	10	None	None	0	0	0
21	1	10	None	None	0	0	0
22	0	11	None	None	1	1	0
23	1	11	None	None	1	1	0
24	0	12	None	None	2	2	0
25	1	12	None	None	2	2	0
26	0	13	None	None	3	3	0
27	1	13	None	None	3	3	0
28	0	14	None	None	4	4	0
29	1	14	None	None	4	4	0
30	0	15	None	None	5	0	1
31	1	15	None	None	5	0	1
32	0	16	None	None	6	1	1
33	1	16	None	None	6	1	1
34	0	17	None	None	7	2	1
35	1	17	None	None	7	2	1
36	0	18	None	None	8	3	1
37	1	18	None	None	8	3	1
38	0	19	None	None	9	4	1
39	1	19	None	None	9	4	1

Figure 2: Figure 2: State decomposition computed by StateCounter for the library in Fig. 1. Each row shows the state of every counter when `seq_counter` is set to a given value (0–39). The `stack` operation creates mutually exclusive branches: when `mut_counter` or `del_counter` is active, the insertion-related counters show `None`, and vice versa. The `product` operation keeps all parent counters active simultaneously—here, `v_counter` cycles through its 2 states for each of the 20 CRE variants.

The following code constructs the counter DAG for the sequence library in Fig. 1. Five leaf counters represent the component dimensions: mutation index, deletion position, insertion site, insertion position, and barcode variant. These are composed using `product` and `stack` to build the full 40-state counter.

```
from statecounter import Manager, Counter, product, stack

# Initialize manager
with Manager() as mgr:
```

```
    # Define leaf counters
    mut_counter = Counter(5, name='mut_counter')
    del_counter = Counter(5, name='del_counter')
    ins_site_counter = Counter(2, name='ins_site_counter')
    ins_position_counter = Counter(5, name='ins_position_counter')
    v_counter = Counter(2).named('v_counter')

    # Build composite counters
    ins_counter = product([ins_position_counter, ins_site_counter],
        name='ins_counter')
    cre_counter = stack([mut_counter, del_counter, ins_counter],
        name='cre_counter')
    seq_counter = product([v_counter, cre_counter],
        name='seq_counter')

# Print DAG
seq_counter.print_dag('minimal')

# Print sequences
seq_counter.get_iteration_df()
```

The `print_dag()` method displays the counter hierarchy. The `cre` counter stacks three alternatives (5 mutations + 5 deletions + 10 insertions = 20 states), and `seq` takes the product with the 2-state barcode counter, yielding 40 total states.

```
seq_counter (n=40)
└── [Product]
    ├── v_counter (n=2)
    └── cre_counter (n=20)
        └── [Stack]
            ├── mut_counter (n=5)
            ├── del_counter (n=5)
            └── ins_counter (n=10)
                └── [Product]
                    ├── ins_position_counter (n=5)
                    └── ins_site_counter (n=2)
```

Research Impact Statement

StateCounter is the computational engine that enables PoolParty to generate sequences in complex DNA libraries on demand. A PoolParty library typically contains sequences constructed through different combinations of operations—some sequences might have point mutations, others deletions, others insertions—and each operation has internal parameters controlling specifics like mutation positions or insertion sites. When PoolParty needs to generate the nth sequence, it must determine two things: which combination of operations to apply, and what parameter values to use for each. StateCounter answers both questions by decomposing n into the states of individual counters. Active counters indicate which operations are in effect; their state values specify the parameters. This decomposition enables PoolParty to generate any sequence directly from its index, without iterating through preceding sequences.

This capability is essential because DNA library design often requires combining sequences generated through fundamentally different processes into a single library. The state decomposition also supports systematic sequence naming, where each name encodes its combinatorial origin (e.g., `del_2.v0` for the second deletion variant with the first barcode), and enables researchers to trace experimental results back to the specific choices that generated each sequence.

DRAFT

Beyond DNA library design, StateCounter addresses a general need in any domain involving complex combinatorial structures: experimental design matrices, parameter sweeps in simulations, combinatorial testing of software configurations, and procedural content generation. We anticipate that StateCounter will find applications wherever researchers need to decompose flat indices into meaningful component states.

AI Usage Disclosure

Z.L. and J.B.K. conceived the project and designed the software architecture. Code was written in Cursor by J.B.K. with iterative assistance from Claude Opus 4.5. Tests and documentation were written by Opus 4.5 with human editing. Z.L. and J.B.K. wrote the paper using Claude Opus 4.5 for help with the initial draft. Z.L. and J.B.K. bear full responsibility for the text of the paper, the documentation, and the content of the software.

Acknowledgements

This work was supported by NIH grant R01 HG011787.

References

Harris, C. R., Millman, K. J., Walt, S. J. van der, & others. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

PLACEHOLDER. (PLACEHOLDER). PLACEHOLDER - combinatorial testing survey or foundational paper. *PLACEHOLDER*.

Python Software Foundation. (2024). *Itertools — functions creating iterators for efficient looping*. Python Standard Library Documentation. https://docs.python.org/3/library/itertools.html

DRAFT